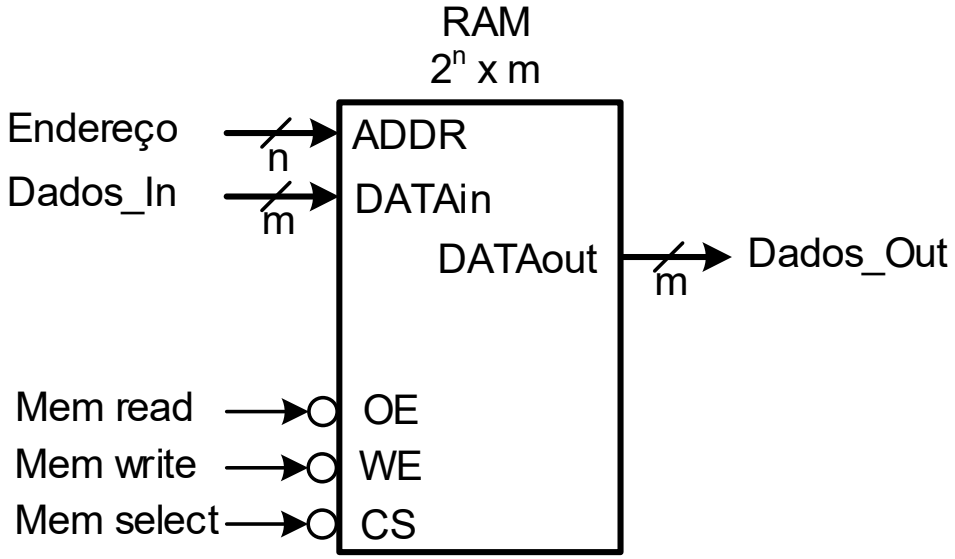
Memória,
Controlo Microprogramado
Circuitos de Transferência de Dados

Memória

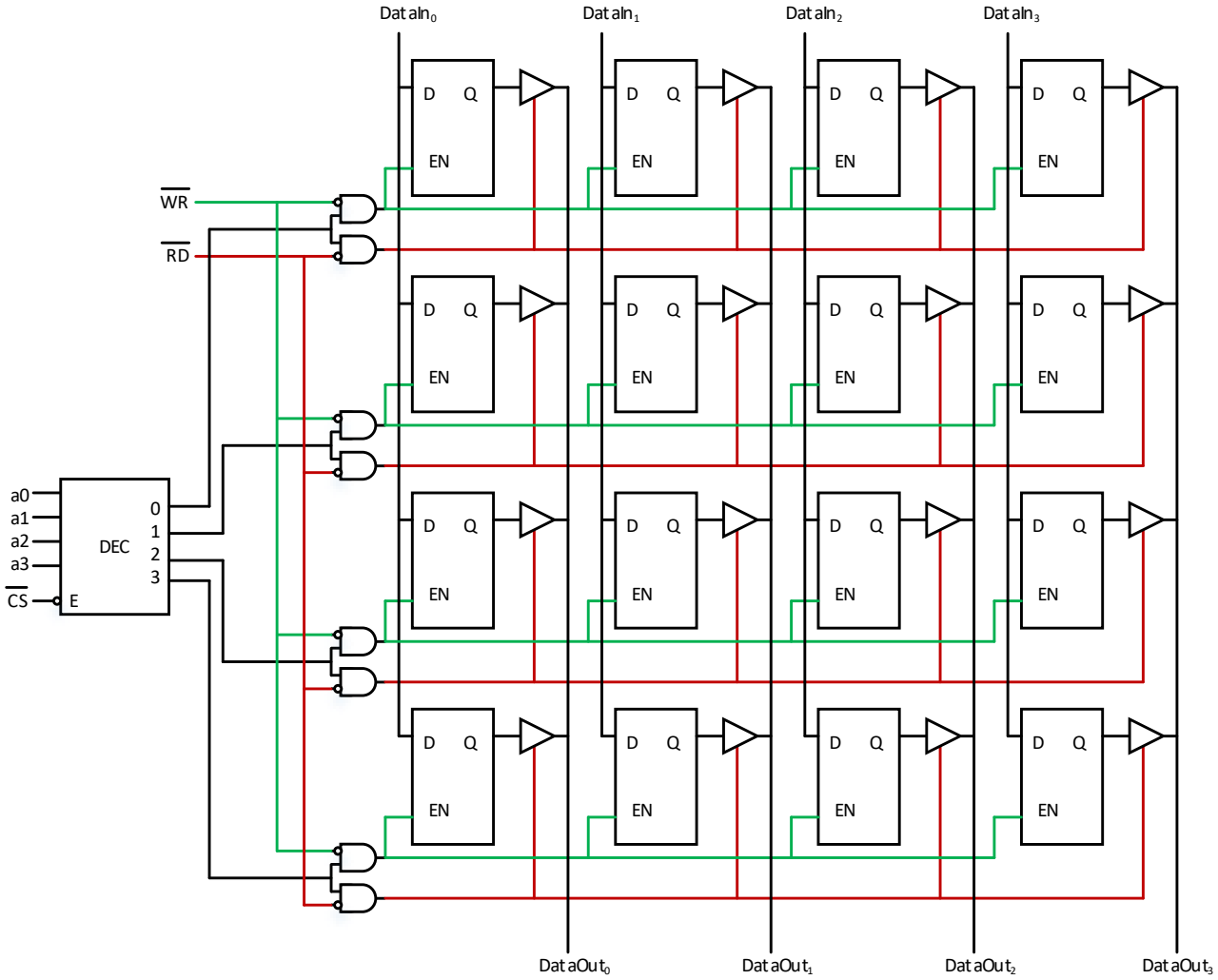
- Memória é dispositivo que permite guardar dados de forma temporária ou permanente;
- Existem dois tipos de memória:
  - Volátil: perdem os dados quando não estão alimentadas;
  - Não volátil: não perdem os dados mesmo quando não estão alimentadas.
- Memória volátil:
  - RAM – *Random Access Memory:*
    - *Estática (SRAM): Mais cara, mas mais rápida;*
    - *Dinâmica (DRAM): Mais barata, mas mais lenta.*
  - *Outras – FF, Registo.*
- *Memória não volátil:*
  - *ROM* – Read Only Memory
  - *Outras – Flash ROM, EEPROM, etc.*

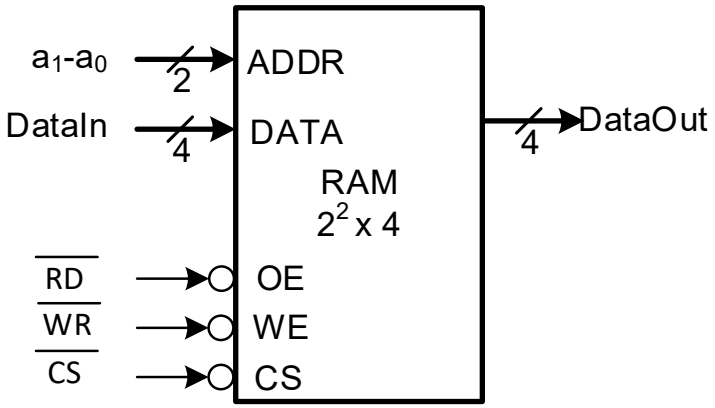
Memória RAM

- **Endereço**: indica o valor da posição da memória onde serão escritos ou lidos os dados;
- **Dados**: Dados a serem lidos ou guardados na memória;
- **Write enable** (WE): sinal de controlo que indica a escrita de dados;
- **Output enable** (OE): sinal de controlo que indica a leitura de dados:
- ***Chip select*** (CS): sinal de controlo que ativa o funcionamento da RAM. Quanto inativo, a RAM coloca as saídas de dados em alta impedância.



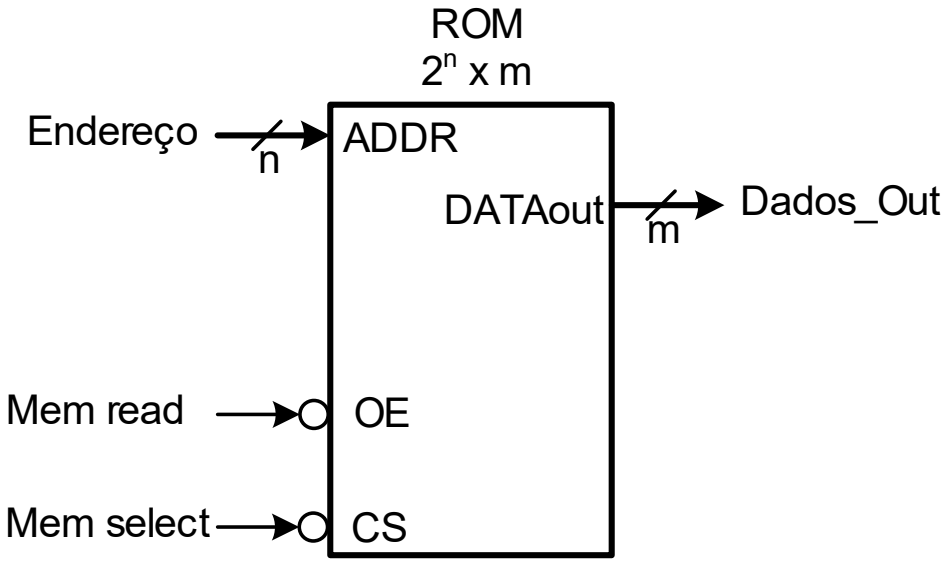
Arquitetura da Memória RAM





Memória ROM

- **Endereço**: indica o valor da posição de memória de onde serão lidos os dados;
- **Dados**: saída de dados;
- **Output enable** (OE): sinal de controlo que indica a leitura de dados:
- ***Chip select*** (CS): sinal de controlo que ativa o funcionamento da ROM. Quanto inativo, a ROM coloca as saídas de dados em alta impedância.



Implementação de Funções com Memória

A memória permite implementar funções lógicas;

- Com memória ROM: funções lógicas não podem ser alteradas;
- Com memória RAM: funções lógicas podem ser alteradas;

Para implementar *n* funções com *m* variáveis numa ROM:

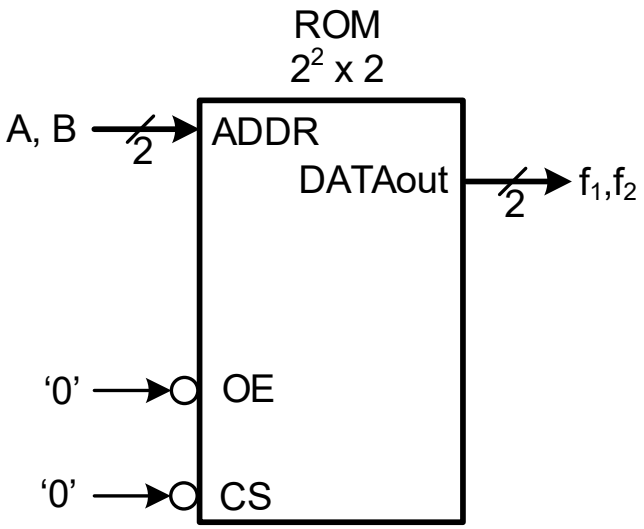
- Ligam-se as *m* entradas da função a *m* linhas de endereço;
- Ligam-se *n* saídas de dados da memória às *n* saídas das funções;

Implementação de Funções com ROM

Exemplo: pretende-se implementar as funções f1(A,B) e f2(A,B) com uma memória ROM.

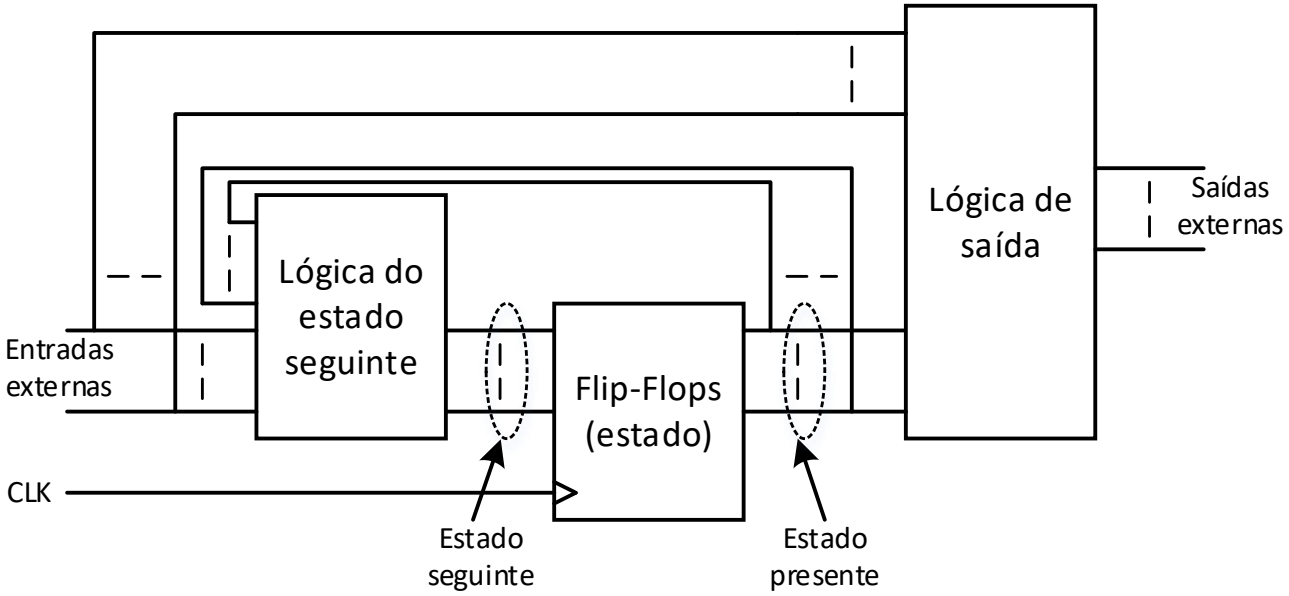
A B	f1	f2
0 0	0	1
0 1	0	0
1 0	1	1
1 1	0	1

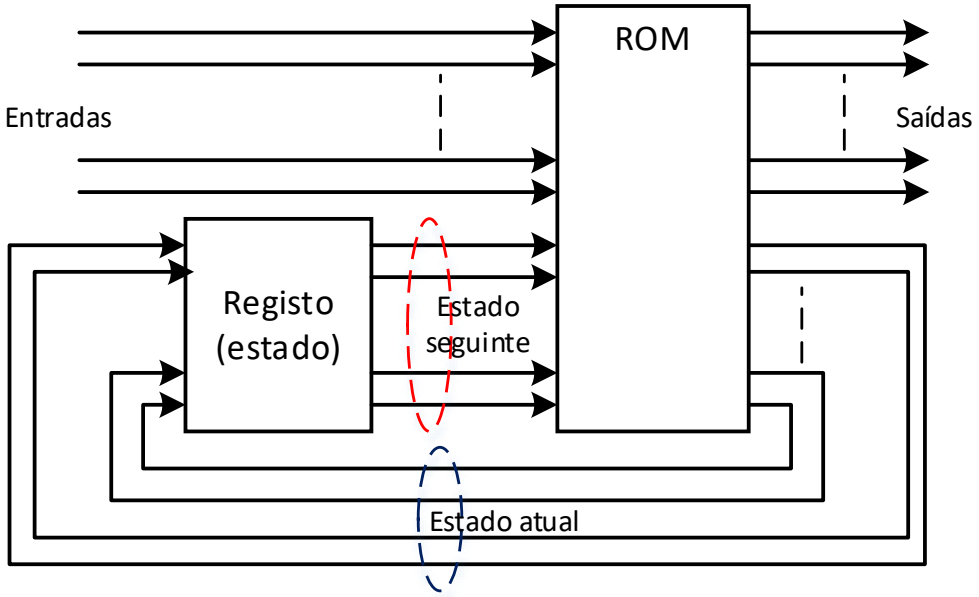
É necessária uma ROM com 2^2 x 2.



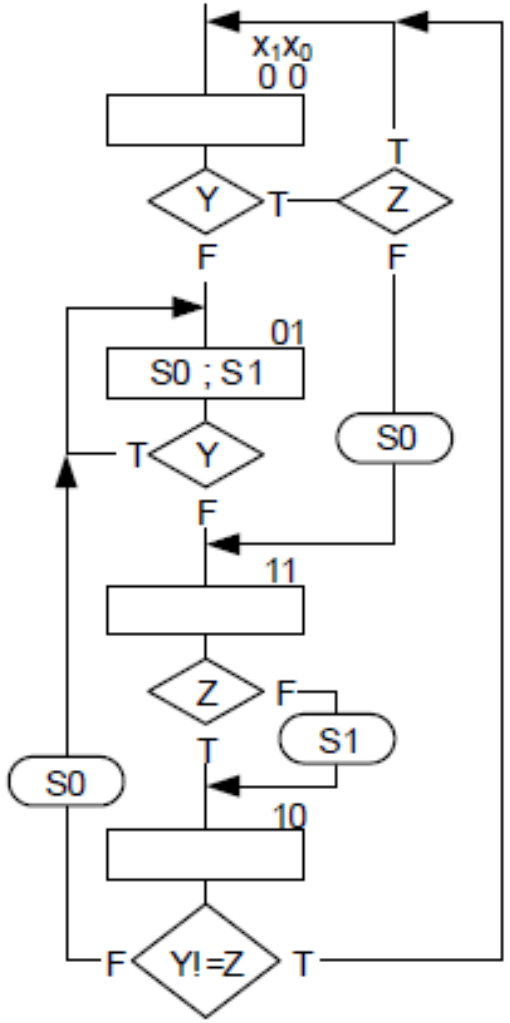
ADDR	DATAout
0 0	0 1
0 1	0 0
1 0	1 1
1 1	0 1

Controlo Microprogramado





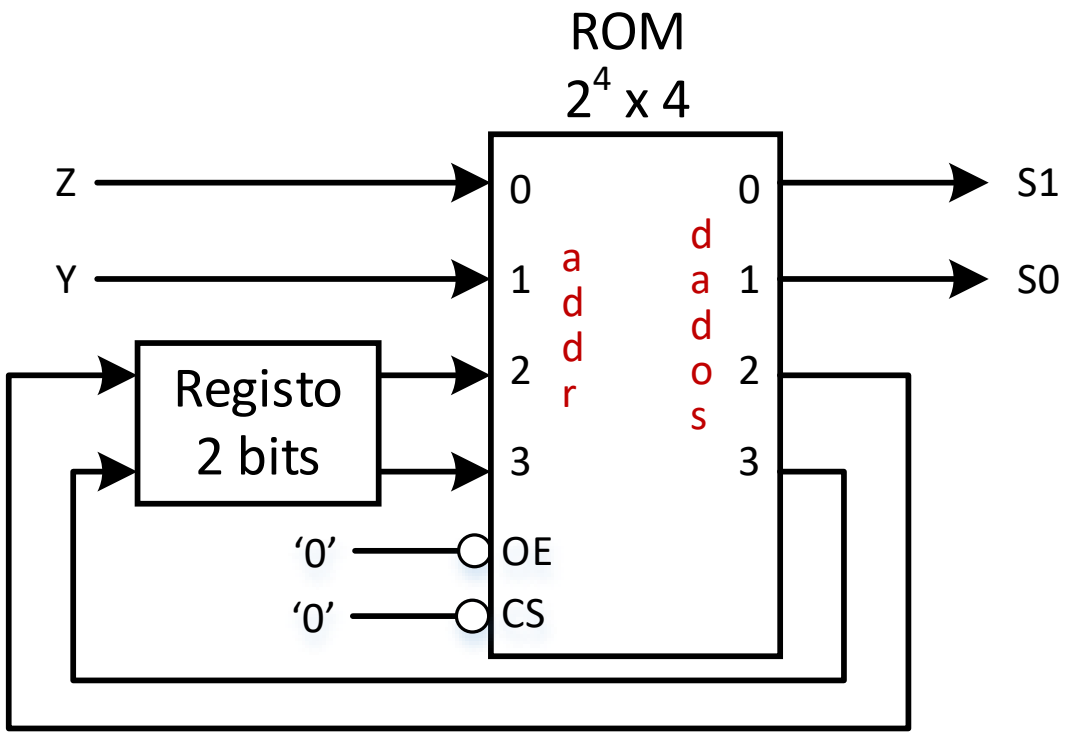
Controlo Microprogramado - Exemplo



EP X1	EP X0	Y	Z	ES		S1	S0
0	0	0	0	0	1	0	0
0	0	0	1	0	1	0	0
0	0	1	0	1	1	0	1
0	0	1	1	0	0	0	0
0	1	0	0	1	1	1	1
0	1	0	1	1	1	1	1
0	1	1	0	0	1	1	1
0	1	1	1	0	1	1	1
1	0	0	0	0	1	0	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	0	0
1	0	1	1	0	1	0	1
1	1	0	0	1	0	1	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	0
1	1	1	1	1	0	0	0

Controlo Microprogramado - Exemplo

Endereço				Dados			
EP		Y	Z	ES		S1	S0
0	0	0	0	0	1	0	0
0	0	0	1	0	1	0	0
0	0	1	0	1	1	0	1
0	0	1	1	0	0	0	0
0	1	0	0	1	1	1	1
0	1	0	1	1	1	1	1
0	1	1	0	0	1	1	1
0	1	1	1	0	1	1	1
1	0	0	0	0	1	0	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	0	0
1	0	1	1	0	1	0	1
1	1	0	0	1	0	1	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	1	0
1	1	1	1	1	0	0	0



Controlo com Lógica Discreta - VHDL

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY control IS
PORT(	RESET : in STD_LOGIC;
	CLK : in std_logic;
	X, Y : in std_logic;
	S0, S1 : OUT STD_LOGIC);
END control;

ARCHITECTURE logicFunction OF control IS

component FFD
PORT(	CLK : in std_logic;
	RESET : in STD_LOGIC;
	D : IN STD_LOGIC;
	EN : IN STD_LOGIC;
	Q : out std_logic);
END component;

signal D0, D1, Q0, Q1 : std_logic;

BEGIN

U0: FFD port map(clk, reset, D0, '1', Q0);

U1: FFD port map(clk, reset, D1, '1', Q1);

-----------------------------------------
-- Implementação do controlo com lógica --
-----------------------------------------
D0 <= ;

D1 <= ;

S0 <= ;

S1 <= ;

END LogicFunction;
```

Controlo com ROM - VHDL

```
ENTITY control IS
PORT(       RESET : in STD_LOGIC;
            CLK : in std_logic;
            X, Y : in std_logic;
            S0, S1 : OUT STD_LOGIC);
END control;

ARCHITECTURE logicFunction OF control IS
component FFD
PORT(       CLK : in std_logic;
            RESET : in STD_LOGIC;
            D : IN STD_LOGIC;
            EN : IN STD_LOGIC;
            Q : out std_logic);
END component;

component ROMcomb
port (      addr : in std_logic_vector(3 downto 0);
            q : out std_logic_vector(3 downto 0));
end component;

signal D0, D1, Q0, Q1 : std_logic;
signal romOut : std_logic_vector(3 downto 0);
signal ROMaddr : std_logic_vector(3 downto 0);
```

```
BEGIN

U0: FFD port map(clk, reset, D0, '1', Q0);

U1: FFD port map(clk, reset, D1, '1', Q1);

ROMaddr(0) <= X;
ROMaddr(1) <= Y;
ROMaddr(2) <= Q0;
ROMaddr(3) <= Q1;

U2: ROMcomb port map(ROMaddr, romOut);

S0 <= romOut(0);
S1 <= romOut(1);
D0 <= romOut(2);
D1 <= romOut(3);



END LogicFunction;
```

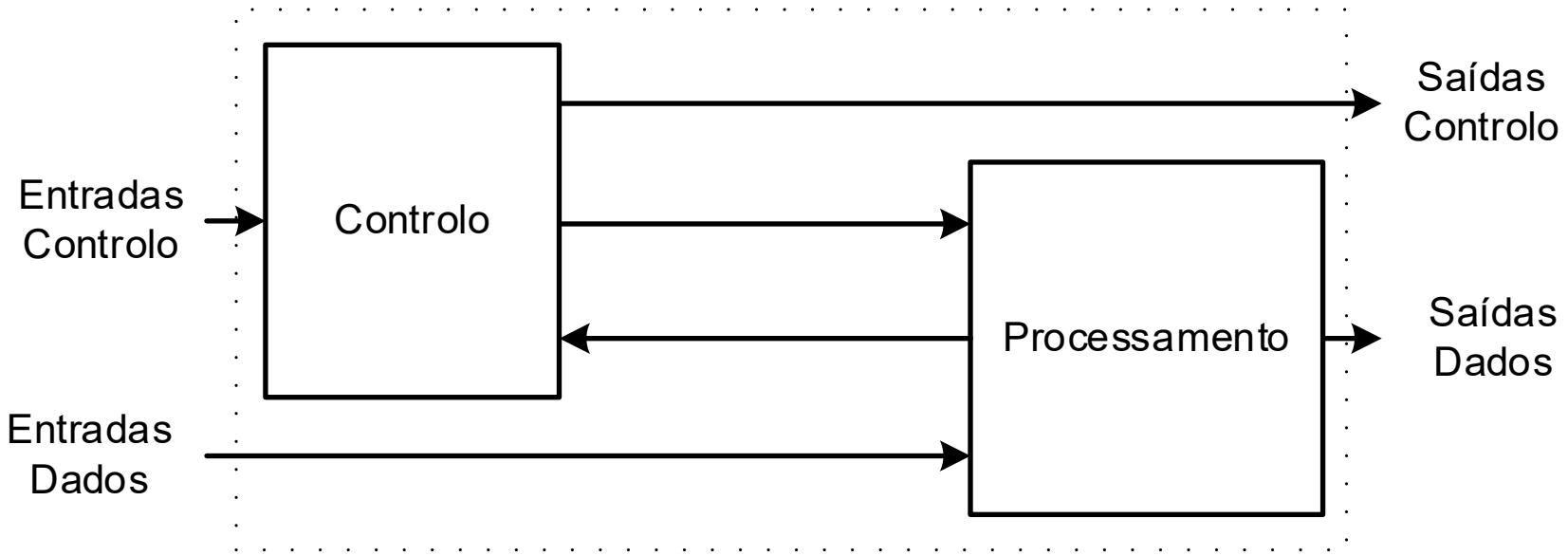
ROM - VHDL

```
entity ROMcomb is
port (        addr : in std_logic_vector(3 downto 0);
              q : out std_logic_vector(3 downto 0));
end ROMcomb;

architecture memory of ROMcomb is

subtype word_t is std_logic_vector(3 downto 0);
type memory_t is array(0 to 15) of word_t;

signal rom : memory_t := (
              "0010", "0010", "0111", "0111",
              "0100", "1000", "0100", "1000",
              "0000", "0000", "1000", "1000",
              "0000",  "0000", "0000", "0000");
begin

              q <= rom(to_integer(unsigned(addr)));
end memory;
```

Circuito de Transferência de Dados

Circuito Processamento/Controlo

- Dois módulos principais:
  - Módulo de processamento;
  - Módulo de controlo



Exemplo – Multiplicação com Somas Sucessivas

Pretende-se implementar um circuito de multiplicação por somas sucessivas: P = A x B = A + A + A + …. + A (B vezes)

Sequência de operações:

Carrega operandos

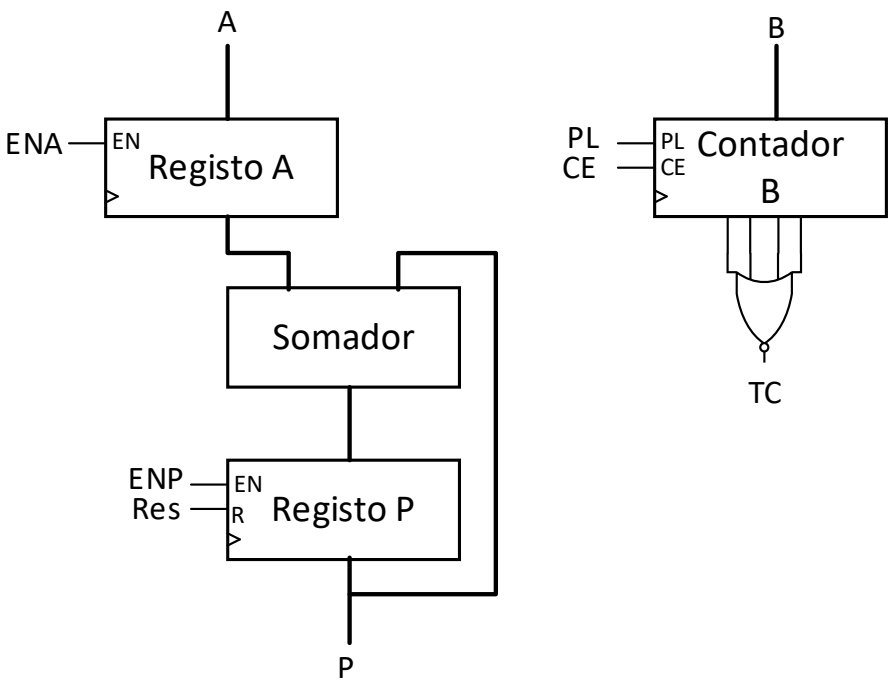
Coloca registo P a 0

Enquanto Z = 0:

Decrementa B

P = P + A

Processamento



Multiplicação com Somas Sucessivas - Controlo

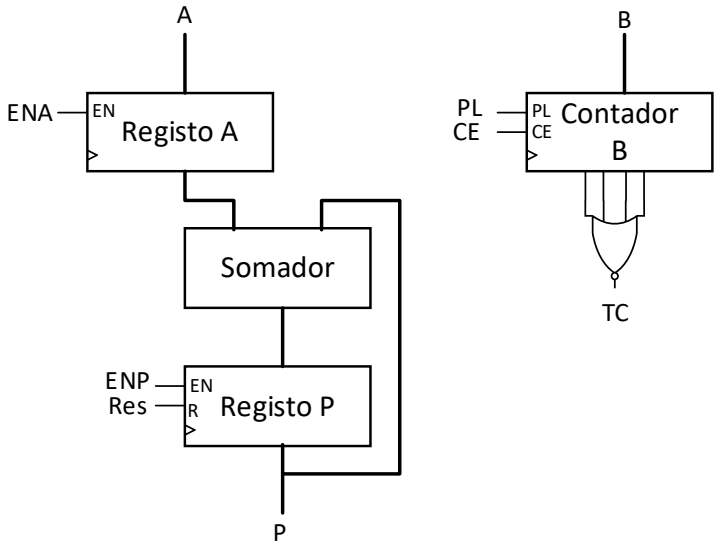
Sequência de operações:

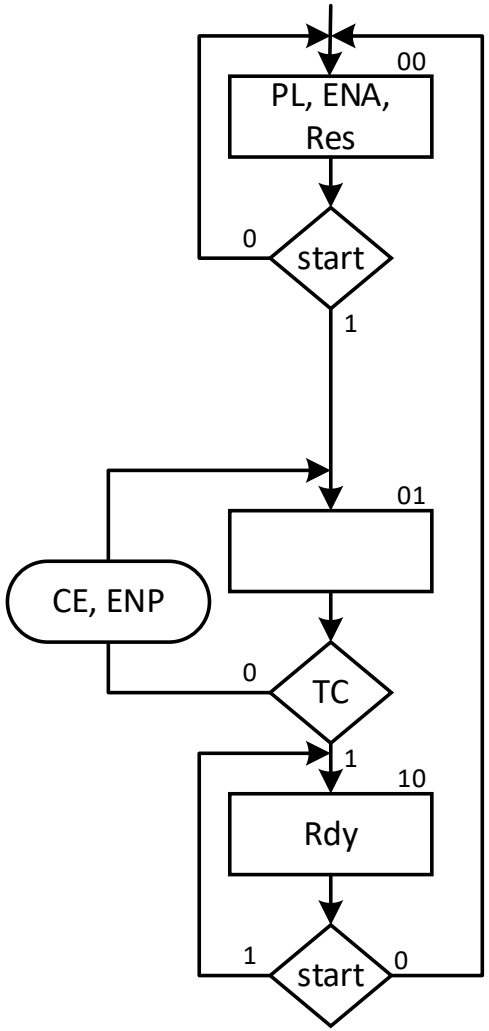
Colocar registo P a 0

Enquanto Z = 0:

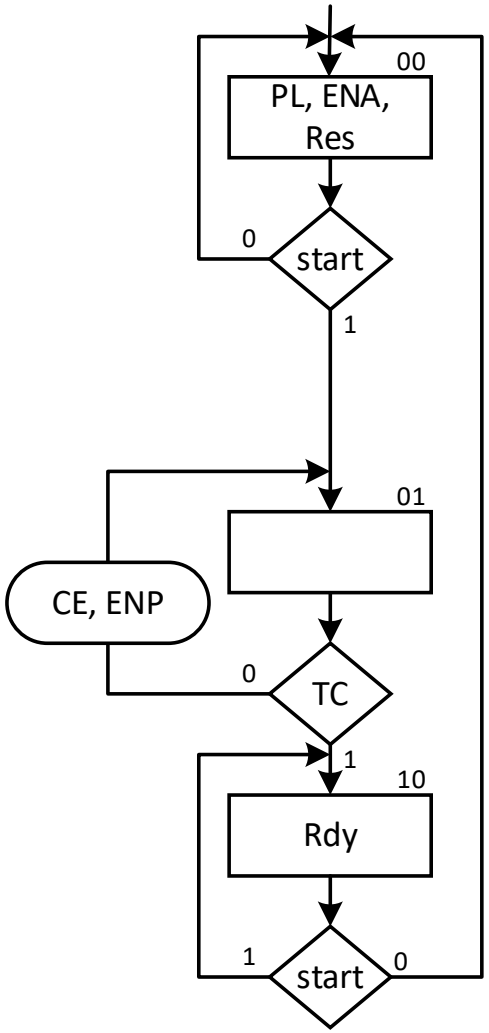
Decrementa B

P = P + A





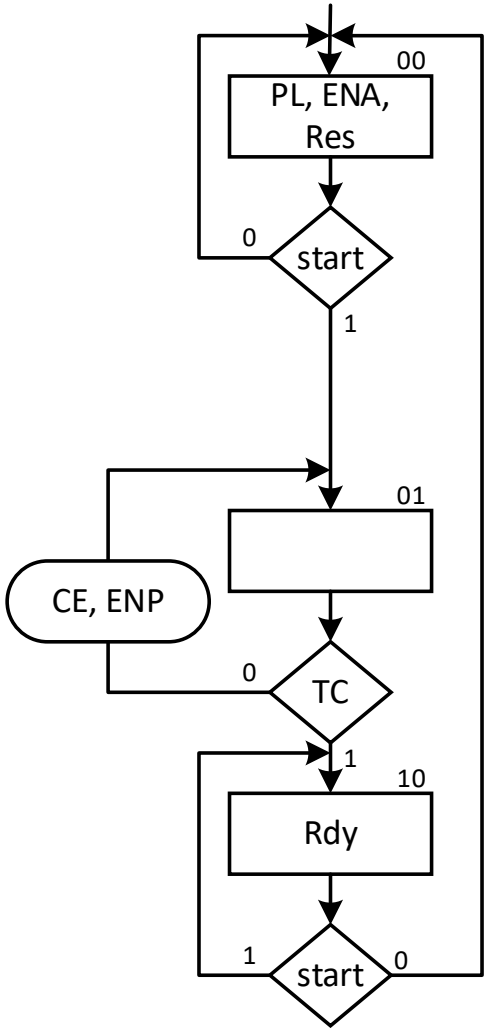
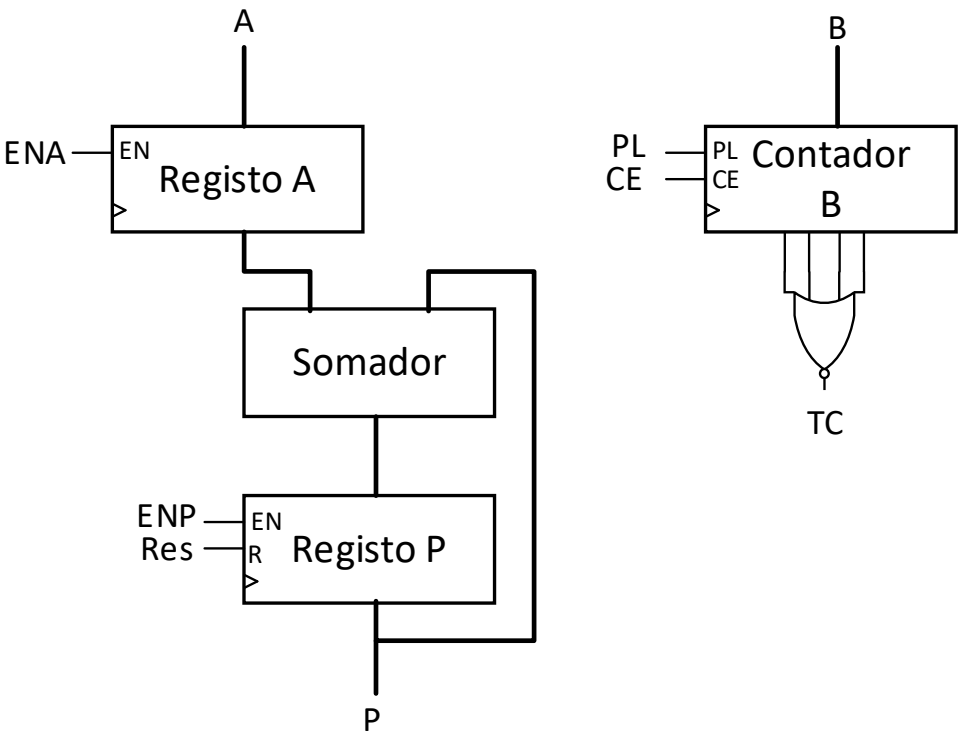
Projeto do Controlo com Lógica Discreta



EP start TC Q1 Q0	ES	PL, Res, ENA	CE, ENP	Rdy
0 0 0 0	0 0	1	0	0
0 0 0 1	0 0	1	0	0
0 0 1 0	0 1	1	0	0
0 0 1 1	0 1	1	0	0
0 1 0 0	0 1	0	1	0
0 1 0 1	1 0	0	0	0
0 1 1 0	0 1	0	1	0
0 1 1 1	1 0	0	0	0
1 0 0 0	0 0	0	0	1
1 0 0 1	0 0	0	0	1
1 0 1 0	1 0	0	0	1
1 0 1 1	1 0	0	0	1

Projeto do Controlo com Lógica Discreta

EP Q1	Q0	start	TC	ES		PL	Res	CE	ENP	Rdy
0	0	0	0	0	0	1	0	0	0	0
0	0	0	1	0	0	1	0	0	0	0
0	0	1	0	0	1	1	0	0	0	0
0	0	1	1	0	1	1	0	0	0	0
0	1	0	0	1	0	0	1	0	0	0
0	1	0	1	1	0	0	1	0	0	0
0	1	1	0	1	0	0	1	0	0	0
0	1	1	1	1	0	0	1	0	0	0
1	0	0	0	1	0	0	0	1	1	0
1	0	0	1	1	1	0	0	1	1	0
1	0	1	0	1	0	0	0	1	1	0
1	0	1	1	1	1	0	0	1	1	0
1	1	0	0	0	0	0	0	0	0	1
1	1	0	1	0	0	0	0	0	0	1
1	1	1	0	1	1	0	0	0	0	1
1	1	1	1	1	1	0	0	0	0	1

```
D0 <= (not q1 and not q0 and start) or
      (not q1 and q0 and (not TC));

D1 <= (not q1 and q0 and TC) or
      (q1 and not q0 and Start);

PL <= not q1 and not q0;
Res <= not q1 and not q0 and start;
CE <= not q1 and q0 and not TC;
Rdy <= q1 and not q0;
```

Circuito de Dados/Controlo